Day 5 - Design Patterns

Patterns are a shared vocabulary for recurring design problems, and games lean on them heavily: factories spawning entities, a single audio manager, a façade over a subsystem, a visitor walking a scene graph. The skill isn't memorising patterns; it's knowing *when each helps and when it hurts*. Today you implement four and, just as importantly, argue when **not** to use them.

Learning objectives

By the end of today you can:

- [ ] Implement Factory Method and say what it decouples.
- [ ] Explain how Singleton works, its real costs (global state, testing), and what to use instead.
- [ ] Implement Facade to simplify a messy subsystem behind one interface.
- [ ] Implement Visitor and explain the double-dispatch it gives you.
- [ ] Argue when to use vs avoid each, with a game-relevant example.

Concepts

Factory Method

Create objects by kind without naming the concrete class at the call site, so adding a type doesn't touch the callers. Ideal for spawning entities/items from data. **Avoid when:** a plain constructor would do.

```
std::unique_ptr<Entity> makeEntity(const std::string& kind){
    if (kind == "orc")  return std::make_unique<Orc>();
    if (kind == "bat")  return std::make_unique<Bat>();
    if (kind == "boss") return std::make_unique<Boss>();
    return nullptr;
}
// Call site never names Orc/Bat/Boss:
auto e = makeEntity(row.kind);
```

❌ **Common Mistake:** a "factory" that the caller then downcasts. If callers still need the concrete type, the factory bought you nothing but a layer.

✅ **Good Practice:** return the base type and put the differing behaviour behind a virtual method, so the call site never names `Orc` / `Bat` / `Boss`.

Singleton

Guarantees one instance with a global access point. Tempting for managers (audio, config). **Avoid when:** it's a global in disguise or it blocks testing, which is most of the time. Prefer passing the dependency in.

```
class AudioManager {
public:
    static AudioManager& instance(){   // one instance, lazily created
        static AudioManager s_instance;
        return s_instance;
    }
    AudioManager(const AudioManager&) = delete;
    AudioManager& operator=(const AudioManager&) = delete;
private:
    AudioManager() = default;
};
```

How the three pieces enforce "one instance, ever":

- `static` on `instance()` makes it a static member function: it belongs to the class and is callable as `AudioManager::instance()` with no object. You can't require an object to get *the* object.
- `static` on `s_instance` makes it a function-local static: constructed once, on the first call, and reused by every call after (lazy init, thread-safe since C++11). This line is the only place in the program the constructor runs.
- `AudioManager() = default;` is there because declaring the deleted copy constructor suppresses the compiler-generated default constructor, so it must be brought back explicitly. It's `private`, so outside code can't do `AudioManager a;`, and the deleted copy/assign mean nobody can duplicate the one instance either.

❌ **Common Mistake:** reaching for it to avoid passing a pointer around. Hidden global state makes tests order-dependent and impossible to isolate.

✅ **Good Practice:** pass the dependency in (a reference through the constructor) and keep Singleton for the rare thing that genuinely must be unique.

Facade

One simple interface in front of a fiddly multi-step subsystem, so callers do one call instead of five in the right order. **Avoid when:** it hides control you actually need.

```
// Subsystem: load, decode, upload, bind. Easy to get out of order.
class TextureLoader {                 // the facade
public:
    Texture load(const std::string& path){
        auto bytes = m_file.read(path);
```

```
        auto image = m_decoder.decode(bytes);
        auto id    = m_gpu.upload(image);
        return Texture{id};
    }
private:
    FileSystem m_file;
    Decoder    m_decoder;
    Gpu        m_gpu;
};
```

❌ **Common Mistake:** a facade that just forwards one call to one call. It adds a layer, and a file to maintain, without simplifying anything.

✅ **Good Practice:** introduce a facade only when it collapses a multi-step, order-sensitive sequence into one call.

Visitor

Add new operations over a *stable* set of types without editing those types, via double dispatch (`element->accept(visitor)` then `visitor.visit(*this)`). Good for walking a scene graph / AST. **Avoid when:** the type set changes more often than the operations (every new type touches every visitor).

```
struct Orc; struct Bat;
struct Visitor {
    virtual void visit(Orc&)= 0;
    virtual void visit(Bat&)= 0;
    virtual ~Visitor() = default;
};
struct Entity { virtual void accept(Visitor&)= 0; virtual ~Entity() =
default; };
struct Orc : Entity { void accept(Visitor& v) override{ v.visit(*this); }
};
struct Bat : Entity { void accept(Visitor& v) override{ v.visit(*this); }
};

// New operation (e.g. RenderVisitor, DamageVisitor) without touching
Orc/Bat:
struct DescribeVisitor : Visitor {
    void visit(Orc&) override{ /* ... */ }
    void visit(Bat&) override{ /* ... */ }
};
```

How the two calls give you double dispatch:

- The problem: you hold an `Entity&` and want behaviour that depends on the *concrete* type (`Orc` vs `Bat`), but a C++ virtual call dispatches on only **one** object's runtime type.
- `e->accept(v)` is the first dispatch: the virtual call lands in the concrete class's override (say `Orc::accept`). Inside it, `this` is statically known to be an `Orc`.

- `v.visit(*this)` is the second dispatch: overload resolution picks `visit(Orc&)` at compile time (we're inside `Orc::accept`), and the virtual call runs the concrete visitor's override of it.
- Net effect: the function that runs is chosen by the pair *(entity type, visitor type)*, with no casts and no `if` chains. A new operation is just a new `Visitor` subclass; `Orc` / `Bat` are never edited.

❌ **Common Mistake:** using Visitor when your hierarchy still grows. Every new type forces an edit to every visitor.

✅ **Good Practice:** use Visitor when the *types* are stable and the *operations* keep growing; otherwise a plain virtual method is simpler.

Trade-offs & the GoF reference

Every pattern trades one kind of flexibility for another. Read the Gang of Four entry for each pattern's *intent* and *consequences*. That section, not the class diagram, is the point.

Exercises

The same three entities recur all day: `orc` (hp 40), `bat` (hp 8), and `boss` (hp 300).

Turn-in lists are exact, no definitions in a header except genuine one-liners. A class listed only as `.hpp` is header-only because every method it has *is* a one-liner, written in the class body. Anything with a loop or more than one statement gets a `.hpp/.cpp` pair.

ex00 - Factory Method

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Spawn `Entity` subclasses from a list of kind strings through a factory returning `std::unique_ptr<Entity>`, with a spawning loop that never names a concrete class.*
📁	**Turn in directory**	`ex00/`
❗	**Turn in files**	**Entity.hpp, Orc.hpp, Bat.hpp, Boss.hpp, EntityFactory.hpp/.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Factory Method, returning polymorphic objects by `unique_ptr<Entity>`, virtual dispatch, keeping call sites free of concrete type names.*
🔍	**Search keywords**	*C++ factory method pattern, unique_ptr make_unique polymorphism, virtual describe, decouple object creation*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<memory>`

- **Goal:** Give `Entity` a pure-virtual `describe()` and three subclasses (`Orc` → `"Orc (hp 40)"`, `Bat` → `"Bat (hp 8)"`, `Boss` → `"Boss (hp 300)"`). Read the kind list `{"orc", "bat", "boss", "bat"}` and spawn the matching subclass through `EntityFactory::make(kind)`, which returns `std::unique_ptr<Entity>`. The spawning loop calls only `make` and `describe`; it never names `Orc` / `Bat` / `Boss`. Binary name: `spawn`.
- **Done when:** The output matches below, and adding a fourth kind (`rat`) would require editing **only** the factory, leaving the spawning loop untouched.

The factory is the *one* place concrete types are named; the call site stays generic:

```
// Entity.hpp
class Entity {
public:
    virtual std::string describe() const= 0;
    virtual ~Entity() = default;
};

// EntityFactory.cpp: the ONE place that names concrete types
std::unique_ptr<Entity> EntityFactory::make(const std::string& kind){
    if (kind == "orc")  return std::make_unique<Orc>();
    if (kind == "bat")  return std::make_unique<Bat>();
    if (kind == "boss") return std::make_unique<Boss>();
```

```
    return nullptr;
}

// main.cpp: never names Orc/Bat/Boss
std::vector<std::string> kinds = {"orc", "bat", "boss", "bat"};
for (const auto& kind : kinds) {
    auto e = EntityFactory::make(kind);
    std::cout << "spawned " << e->describe() << '\n';
}
```

Output example:

```
$ ./spawn
spawned Orc (hp 40)
spawned Bat (hp 8)
spawned Boss (hp 300)
spawned Bat (hp 8)
```

The loop mentions only the kind string and the base `Entity`, so a new type is one new case in the factory and nothing at the call site. That's the decoupling the pattern buys.

ex01 - Facade over a subsystem

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Wrap a fiddly four-step save subsystem (serialize → compress → checksum → write) behind a facade exposing one* `save()` *method that* `main` *uses exclusively.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**GameState.hpp, Serializer.hpp/.cpp, Compressor.hpp/.cpp, Checksum.hpp/.cpp, Writer.hpp/.cpp, SaveGameFacade.hpp/.cpp, main.cpp, Makefile**
🛠️	**Skills required**	*Facade pattern, hiding a multi-step ordered subsystem, exposing a single method, keeping callers off the internals.*
🔍	**Search keywords**	*C++ facade pattern, subsystem wrapper, single entry point, simplify interface*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<fstream>`

- **Goal:** `GameState` is a plain struct of two ints, `level` and `hp`, in its own header. Both `main` and the facade signature need it, so it belongs to neither. Build a `SaveGame` subsystem of four small classes whose steps **must** run in this order: `Serializer` turns a `GameState` into text, `Compressor` shrinks it, `Checksum` computes a byte sum, `Writer` writes the bytes to a file. Wrap all four behind a `SaveGameFacade` exposing a single `save(state, path)` method. `main` calls only `save`. Binary name: `save`.
- **Done when:** `main` performs the whole save in one call, never touches an internal step, and the run prints each stage in order matching below.

`main` knows only the facade, so it cannot run the steps out of order:

```
// GameState.hpp: plain data, no methods, no .cpp
struct GameState { int level; int hp; };

// SaveGameFacade.cpp: one call hides the four ordered steps
void SaveGameFacade::save(const GameState& state, const std::string& path){
    std::string text   = m_serializer.serialize(state);   // 1
    std::string packed = m_compressor.compress(text);     // 2
    unsigned    sum    = m_checksum.of(packed);           // 3
    m_writer.write(path, packed, sum);                    // 4
    std::cout << "save complete\n";
```

```
}

// main.cpp: knows only the facade
SaveGameFacade save;
save.save(GameState{3, 27}, "save.dat");   // one call, correct order
guaranteed
```

Output example:

```
$ ./save
serialize -> level=3;hp=27
compress  -> 3;27 (13 -> 4 bytes)
checksum  -> 0xD7
write     -> save.dat (4 bytes)
save complete
```

`main` never calls serialize/compress/checksum/write itself, so it can't get the order wrong. Enforcing that order is the facade's whole job.

ex02 - Visitor

.	.	.
💢	**Difficulty**	⭐⭐⭐⭐
🧠	**Objectives**	*Over the fixed ex00 entity set, implement two visitors (`DescribeVisitor`, `TotalHpVisitor`) using `accept` / `visit` double dispatch, run both over one container without adding a method to the entity classes.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**Entity.hpp, Orc.hpp, Bat.hpp, Boss.hpp, Visitor.hpp, DescribeVisitor.hpp, TotalHpVisitor.hpp, main.cpp, Makefile**
🛠️	**Skills required**	*Visitor pattern, double dispatch (`accept` then `visit`), adding operations without touching the visited types.*
🔍	**Search keywords**	*C++ visitor pattern double dispatch, accept visit, add operation without modifying class, virtual dispatch two types*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<memory>`

- **Goal:** Copy the ex00 entities (`orc` 40, `bat` 8, `boss` 300) and give `Entity` a `virtual void accept(Visitor&)`. Implement two visitors: `DescribeVisitor` prints each entity, `TotalHpVisitor` accumulates their hp. Run **both** over the same `std::vector<std::unique_ptr<Entity>>`. Binary name: `visit`.
- **Done when:** Both operations run over the one container, no new method is added to `Orc` / `Bat` / `Boss` to support a new operation, and the output matches below (total hp `348`).

```
// Visitor.hpp
class Orc; class Bat; class Boss;   // forward declarations: visit only
takes references
struct Visitor {
    virtual void visit(Orc&)= 0;
    virtual void visit(Bat&)= 0;
    virtual void visit(Boss&)= 0;
    virtual ~Visitor() = default;
};

// Orc.hpp: header-only, so accept is defined in the class body
class Orc : public Entity {
public:
    std::string describe() const override{ return "Orc (hp 40)"; }
    int hp() const{ return 40; }
    void accept(Visitor& v) override{ v.visit(*this); }  // first dispatch;
visit(*this) is the second
};

// TotalHpVisitor: a new operation, zero changes to Orc/Bat/Boss
struct TotalHpVisitor : Visitor {
    int total = 0;
    void visit(Orc& o)  override{ total += o.hp(); }
    void visit(Bat& b)  override{ total += b.hp(); }
    void visit(Boss& b) override{ total += b.hp(); }
};

// main.cpp: one container, driven by two visitors
DescribeVisitor describe;
TotalHpVisitor  totalHp;
for (auto& e : entities) e->accept(describe);
for (auto& e : entities) e->accept(totalHp);
std::cout << totalHp.total << '\n';
```

Output example:

```
$ ./visit
-- describe --
Orc (hp 40)
Bat (hp 8)
Boss (hp 300)
```

```
-- total hp --
348
```

Adding a third operation is a new visitor class, and the entity types are never touched. That's exactly the trade Visitor makes, and why it hurts if your *types* keep changing instead.

ex03 (discussion) - When NOT to

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*In ~1 page, argue when you would refuse Singleton and one other pattern from this day, name a concrete alternative for each, and make the testability cost concrete.*
📁	**Turn in directory**	`ex03/`
❗	**Turn in files**	**NOTES.md**
🛠️	**Skills required**	*Judging when a pattern hurts, dependency injection as the Singleton alternative, articulating the testability cost concretely.*
🔍	**Search keywords**	*singleton anti-pattern testing, dependency injection vs singleton, global state test isolation, when not to use design pattern*
🔮	**Command required**	none (written deliverable, nothing to build)
📚	**Allowed libraries**	none

- **Goal:** In ~1 page, argue when you would **refuse** Singleton and one other pattern from this day, and what you'd do instead (e.g. dependency injection instead of a Singleton audio manager). Reference the testability cost concretely.
- **Done when:** `NOTES.md` names a specific refusal case per pattern and a concrete alternative, not a generic "it depends".

This exercise has no program. Structure `NOTES.md` like this so each claim is concrete:

```
# When NOT to use a pattern

## Singleton, refuse when: <specific case>
Instead: <concrete alternative, e.g. inject AudioManager& via the
constructor>
Testability cost: <e.g. a Singleton audio manager is shared across tests;
a test that mutes audio leaks that state into the next test, so results
depend on run order>

## <second pattern>, refuse when: <specific case>
```

```
Instead: <concrete alternative>
Testability cost: <concrete>
```

A refusal case and a named alternative per pattern is the whole grade. "It depends" scores nothing.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- Patterns build on Day 3 (polymorphism) and Day 2 (ownership); no leaks.
- Polymorphic bases have a `virtual` destructor; follows Day 1 conventions.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-05`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read.
4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.